

DVWA + SafeLine WAF Cybersecurity Homelab

Deployment, Validation, Hardening, and Root Cause Analysis

Final Project Report

Status: COMPLETE

Date: 23 September 2026

Environment: Isolated Home Cybersecurity Lab

Project outcome

A functioning DVWA lab protected by SafeLine WAF was deployed and validated against SQL injection, HTTP-flood/rate-limit behavior, authentication, application authorization, and source-IP deny controls. The DVWA backend was then hardened to loopback-only access to eliminate direct TCP/8080 WAF bypass from the LAN.

Document Map

This report closes the current project scope and records the final architecture, troubleshooting history, test evidence, security hardening, and recommended next improvements. The screenshots are taken from the actual lab sessions documented during implementation.

Section	Topic
1	Executive Summary
2	Project Objectives and Scope
3	Lab Environment and Final Architecture
4	Implementation and Connectivity Setup
5	Root Cause Analysis: dwwa.local Returned 404
6	Security Validation: SQL Injection
7	Security Validation: HTTP Flood / Rate Limiting
8	Security Validation: SafeLine Authentication
9	Security Validation: Source-IP Deny Rule
10	Backend Hardening: Closing the Direct :8080 Bypass
11	Final Validation Matrix
12	Security Findings and Lessons Learned
13	Recommendations and Future Enhancements
14	Project Closure
Appendix A	Key Commands and Evidence

1. Executive Summary

This project built a controlled web-application security lab using Damn Vulnerable Web Application (DVWA), SafeLine WAF, Kali Linux, Apache, Docker, and an existing Traefik/k3s homelab on a single Ubuntu host. The lab was designed to demonstrate both offensive testing and defensive controls while keeping the environment isolated from production systems.

The project began with a connectivity problem: DVWA worked directly at `http://192.168.1.17:8080` but the domain-based HTTPS path returned a 404. Troubleshooting showed that TCP/443 was already owned by Traefik. Verbose TLS output exposed the key indicator, "CN=TRAEFIK DEFAULT CERT", proving that requests intended for SafeLine were terminating at Traefik. SafeLine was therefore moved to HTTPS/8443, while Traefik remained on 443.

After the WAF path was operational, the lab validated SQL injection blocking, attack logging, HTTP-flood/rate-limit enforcement, Anti-Bot recovery, SafeLine authentication and authorization, and source-IP deny rules. The project then went beyond the original guide by remediating a WAF-bypass weakness: Apache was changed from listening on all interfaces at `*:8080` to loopback-only `127.0.0.1:8080`, while SafeLine used its host-networked reverse proxy to reach the backend internally.

Completion status

The current project scope is complete. Remaining items in this report are optional portfolio-grade enhancements rather than blockers to project closure.

2. Project Objectives and Scope

- Deploy DVWA as an intentionally vulnerable web application for controlled testing.
- Protect DVWA with SafeLine WAF using a reverse-proxy architecture.
- Use Kali Linux as the controlled test client and attack workstation.
- Demonstrate WAF detection and prevention for SQL injection.
- Demonstrate HTTP flood / rate limiting and Anti-Bot challenge behavior.
- Enforce SafeLine authentication in front of DVWA and validate application authorization.
- Create and validate a source-IP deny rule.
- Troubleshoot the Traefik/SafeLine HTTPS port conflict using network and TLS evidence.
- Close direct backend access to DVWA so LAN clients cannot bypass the WAF.
- Document evidence, lessons learned, risks, and future architecture improvements.

Out of scope for the current closure: exposing the lab to the public Internet, production deployment, enterprise identity-provider integration, and high-volume denial-of-service testing. All attack traffic was limited to the user-owned homelab.

3. Lab Environment and Final Architecture

Component	Address / Port	Role
Ubuntu host	192.168.1.17	Single physical host for k3s/Traefik, SafeLine, and Apache/DVWA
Kali Linux VM	192.168.1.56	Controlled security testing client
Traefik	192.168.1.17:443	Existing k3s HTTPS ingress
SafeLine WAF	192.168.1.17:8443	Protected DVWA HTTPS entry point
SafeLine management	192.168.1.17:9443	WAF administration interface
Apache / DVWA	127.0.0.1:8080	Final backend listener, loopback-only
Local hostname	dvwa.local -> 192.168.1.17	Kali /etc/hosts resolution

Final request path:

```

Kali Linux
  |
  | HTTPS :8443
  v
SafeLine WAF
  | Auth + WAF inspection
  | HTTP to loopback backend

```

```
v
127.0.0.1:8080
Apache / DVWA

Direct LAN access to 192.168.1.17:8080 -> unavailable
```

Why 8443 is currently used

Port 8443 is an alternate HTTPS listener. It is not cryptographically weaker than 443; TLS configuration determines transport security. Port 443 remains occupied by Traefik on the same host. A future TLS-passthrough design can restore a clean `https://dvwa.local` URL without removing SafeLine.

4. Implementation and Connectivity Setup

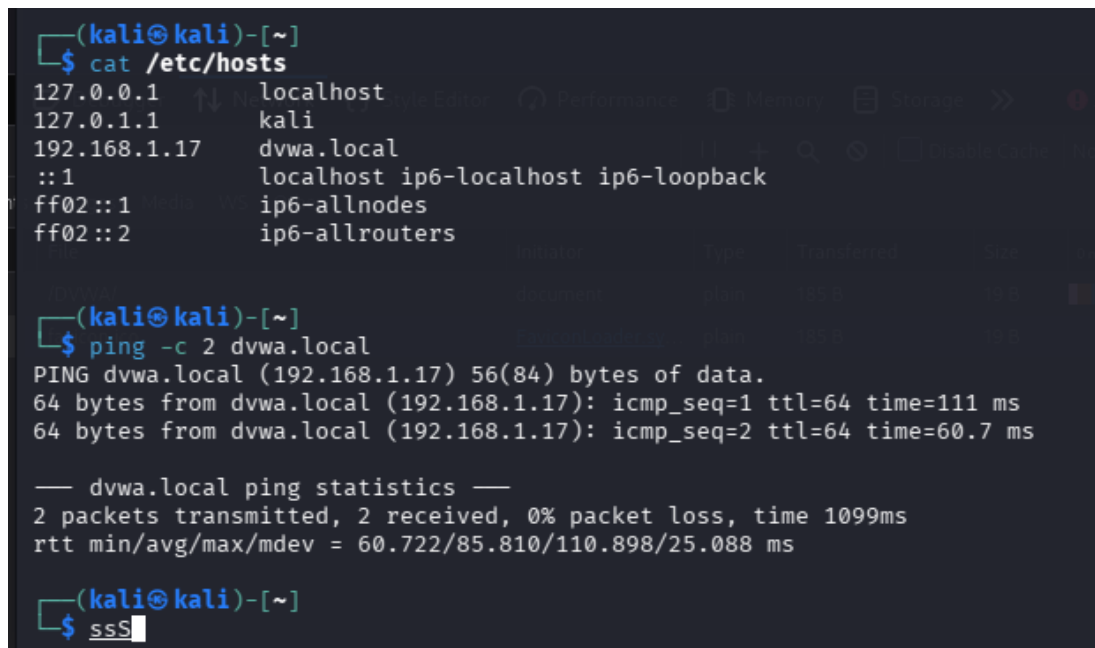
4.1 Direct DVWA baseline

The backend was first validated directly at `http://192.168.1.17:8080/DVWA/login.php`. A direct curl request returned HTTP 200 from Apache, proving that DVWA and Apache were healthy before debugging the domain/WAF path.

```
curl -I http://192.168.1.17:8080/DVWA/login.php
HTTP/1.1 200 OK
Server: Apache/2.4.58 (Ubuntu)
```

4.2 Kali hostname resolution

The Kali client needed a local hosts entry so `dvwa.local` resolved to the Ubuntu server. This corrected the client-side name-resolution issue but did not yet solve the HTTPS 404.



```
(kali㉿kali)-[~]
$ cat /etc/hosts
127.0.0.1    localhost
127.0.1.1    kali
192.168.1.17 dvwa.local
::1          localhost ip6-localhost ip6-loopback
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters

(kali㉿kali)-[~]
$ ping -c 2 dvwa.local
PING dvwa.local (192.168.1.17) 56(84) bytes of data:
64 bytes from dvwa.local (192.168.1.17): icmp_seq=1 ttl=64 time=111 ms
64 bytes from dvwa.local (192.168.1.17): icmp_seq=2 ttl=64 time=60.7 ms

— dvwa.local ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1099ms
rtt min/avg/max/mdev = 60.722/85.810/110.898/25.088 ms

(kali㉿kali)-[~]
$ ssS
```

Figure 1. Kali `/etc/hosts` entry and successful resolution of `dvwa.local` to `192.168.1.17`.

4.3 SafeLine listener moved to 8443

SafeLine was ultimately configured as an HTTPS reverse proxy on TCP/8443. The backend initially pointed to the host on port 8080 and was later hardened to the loopback-only address `127.0.0.1:8080`.

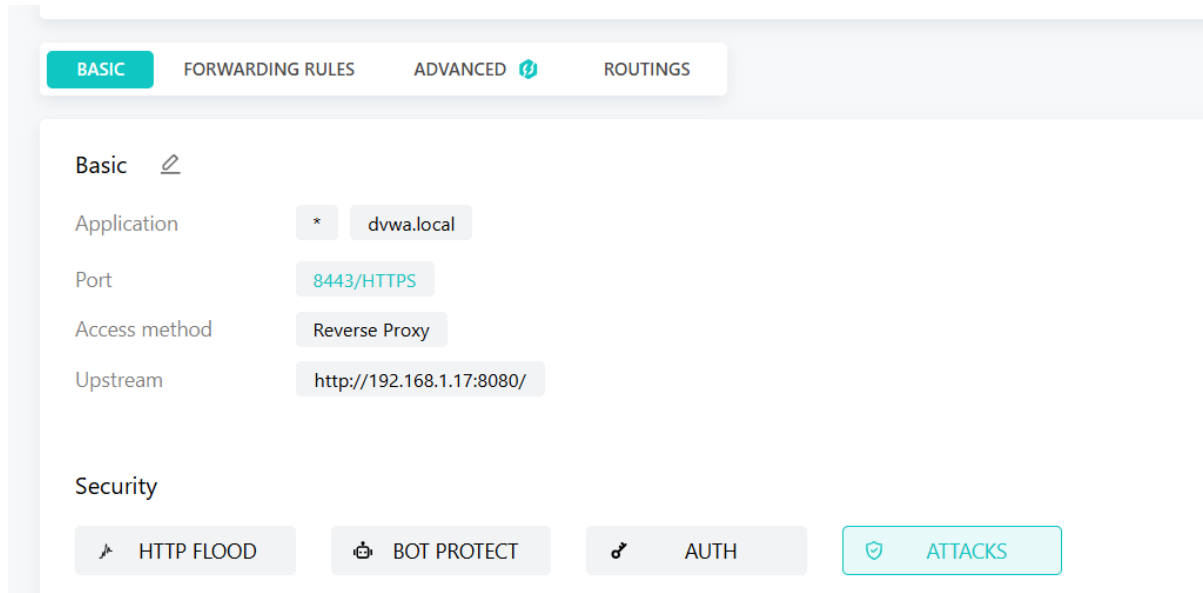


Figure 2. SafeLine application configured on HTTPS/8443 during the working reverse-proxy stage (before final loopback backend hardening).

5. Root Cause Analysis: dvwa.local Returned 404

5.1 Symptom

DVWA was reachable directly on TCP/8080, but browsing to the domain-based HTTPS URL returned a plain 404 page.

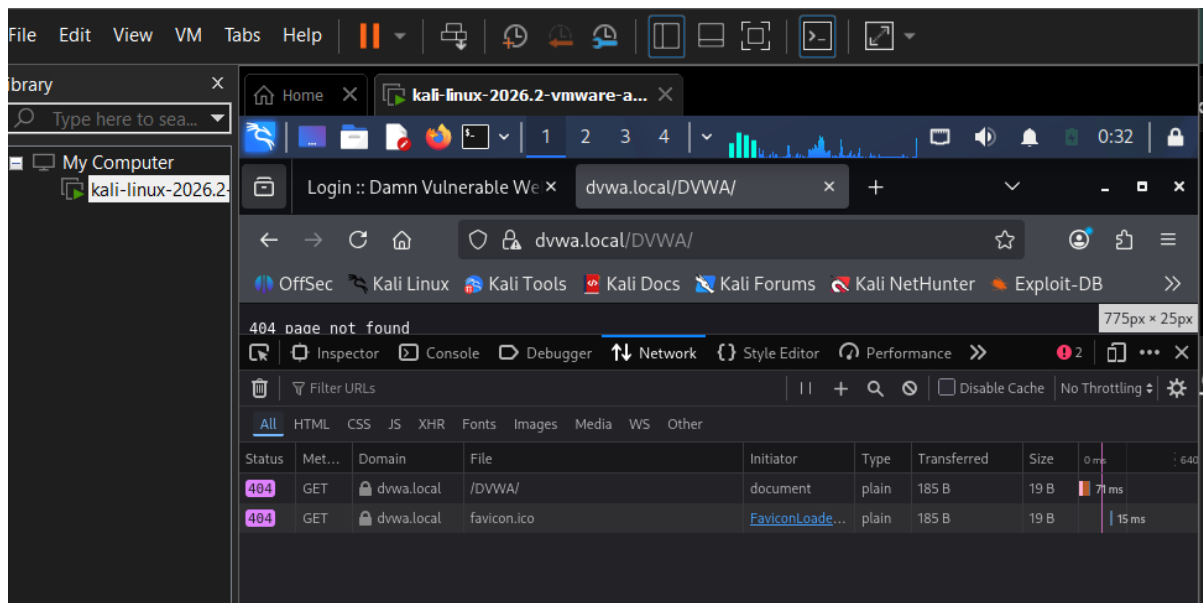


Figure 3. Initial domain request returning a 404 instead of DVWA.

5.2 Key diagnostic evidence

The decisive test was verbose curl against the default HTTPS port. The host resolved correctly, but the TLS certificate identified Traefik rather than SafeLine:

```
curl -k -v https://dvwa.local/DVWA/login.php

* Host dvwa.local:443 was resolved.
* Trying 192.168.1.17:443...
* Server certificate:
*   subject: CN=TRAEFIK DEFAULT CERT
*   issuer: CN=TRAEFIK DEFAULT CERT
< HTTP/2 404
```

404 page not found

Root cause

TCP/443 on 192.168.1.17 was already handled by Traefik. SafeLine was also configured for 443, so requests to <https://dvwa.local> were terminating at Traefik. Because Traefik had no matching dvwa.local route, it returned its default 404 response.

5.3 Corrective action and proof

SafeLine was moved to TCP/8443. The same hostname and path then presented the intended dvwa.local certificate, returned HTTP 200, and issued a SafeLine session cookie. The only material routing variable changed was the destination port, which isolated the conflict to the listener on 443.

```
curl -k -v https://dvwa.local:8443/DVWA/login.php
```

```
* Trying 192.168.1.17:8443...
* Server certificate: ... CN=dvwa.local
< HTTP/1.1 200 OK
< Set-Cookie: sl-session=...
```

RCA conclusion: the incident was caused by a port/listener collision, not by DVWA, Apache, DNS, or the /DVWA path. The earlier missing Kali hosts entry was a contributing issue, but once corrected it exposed the primary TCP/443 conflict.

6. Security Validation: SQL Injection

DVWA was set to Low security and the SQL Injection module was tested through the SafeLine-protected endpoint. A typical boolean-style injection payload was submitted only against the user-owned DVWA instance.

Example lab payload: `1' OR '1'='1' #`

6.1 WAF enforcement

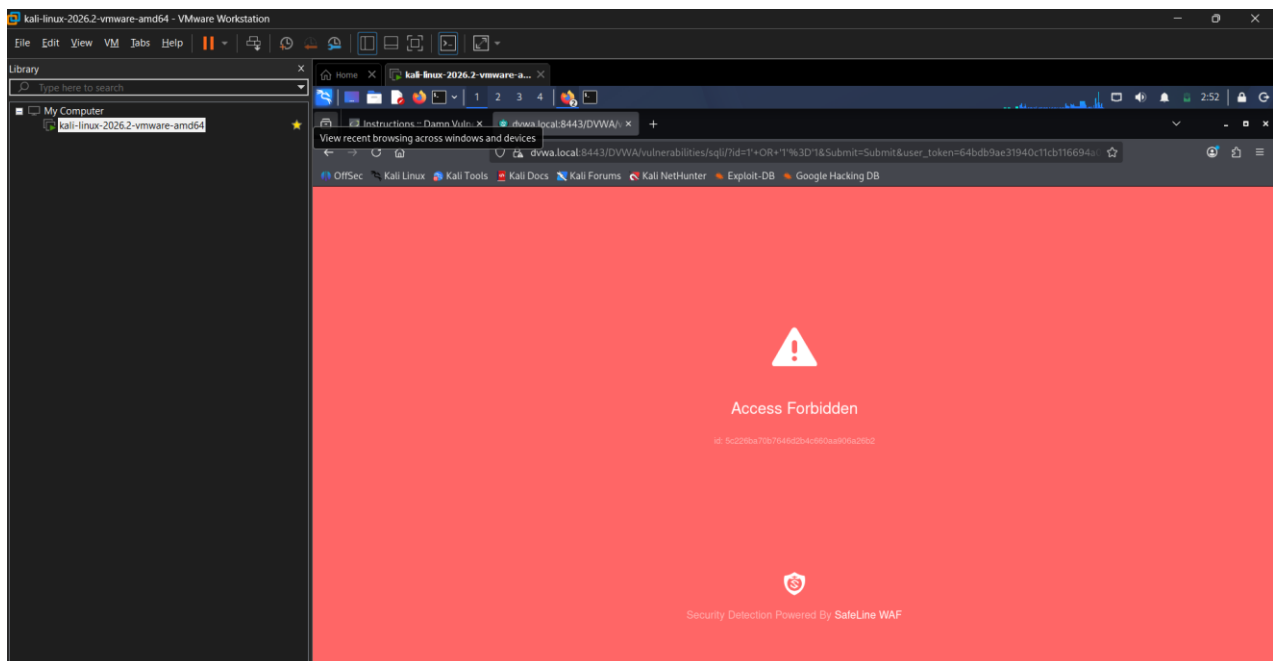


Figure 4. SafeLine returned Access Forbidden when the SQL injection request was submitted through the protected endpoint.

6.2 Statistics and attack logging

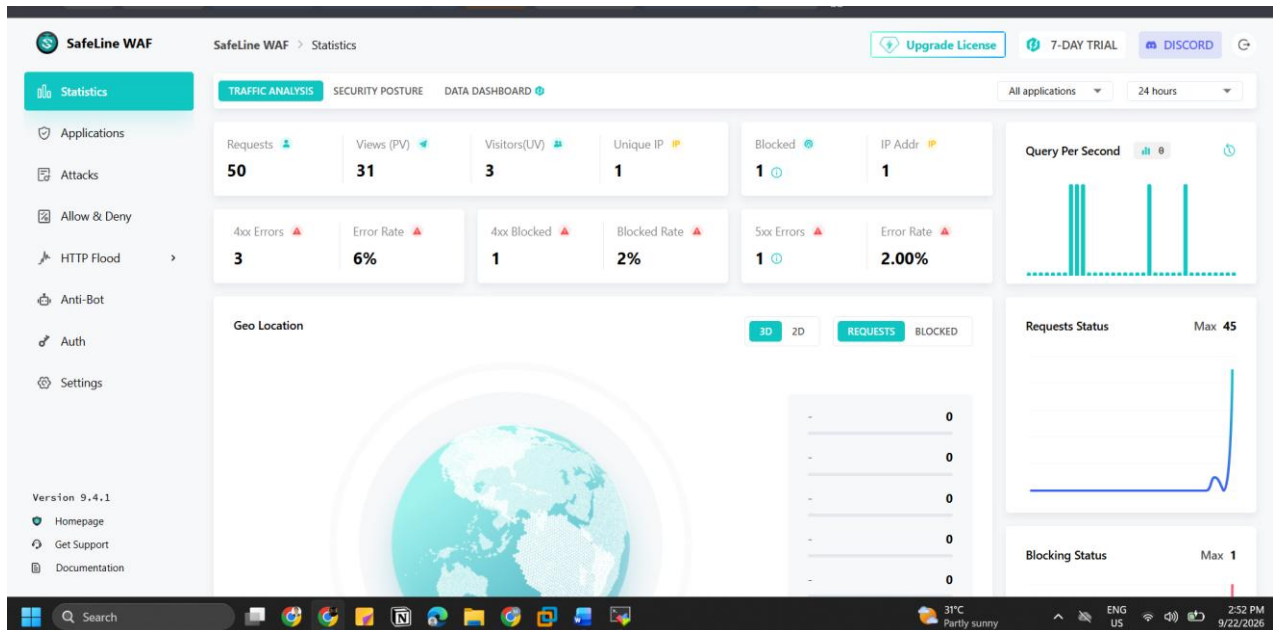


Figure 5. SafeLine statistics showing blocked traffic after the SQL injection test.

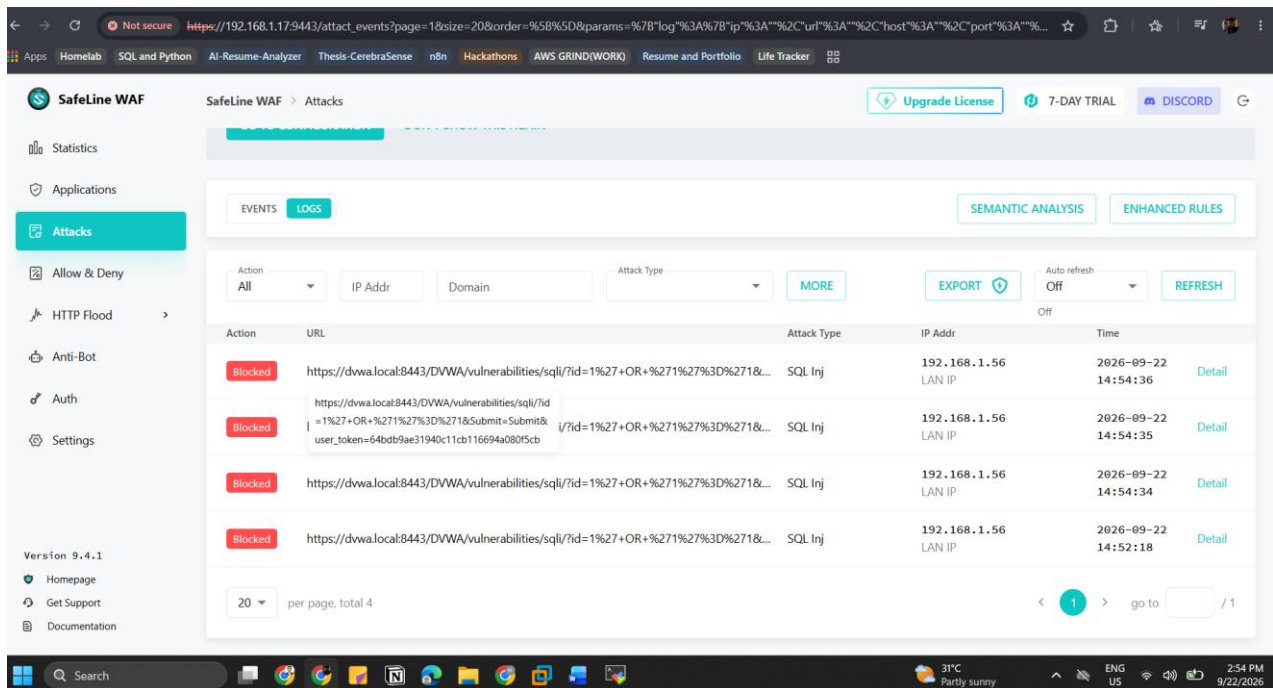


Figure 6. SafeLine attack logs showing multiple blocked SQL injection requests from Kali (192.168.1.56).

Result

PASS - SafeLine detected the SQL injection request, blocked it before normal DVWA processing, and produced attack-log evidence tied to the Kali source IP.

7. Security Validation: HTTP Flood / Rate Limiting

SafeLine Basic Access Limit was used for a controlled burst test. The final lab test used a deliberately low threshold so enforcement could be demonstrated without generating excessive traffic. Challenge mode was selected rather than a long hard block.

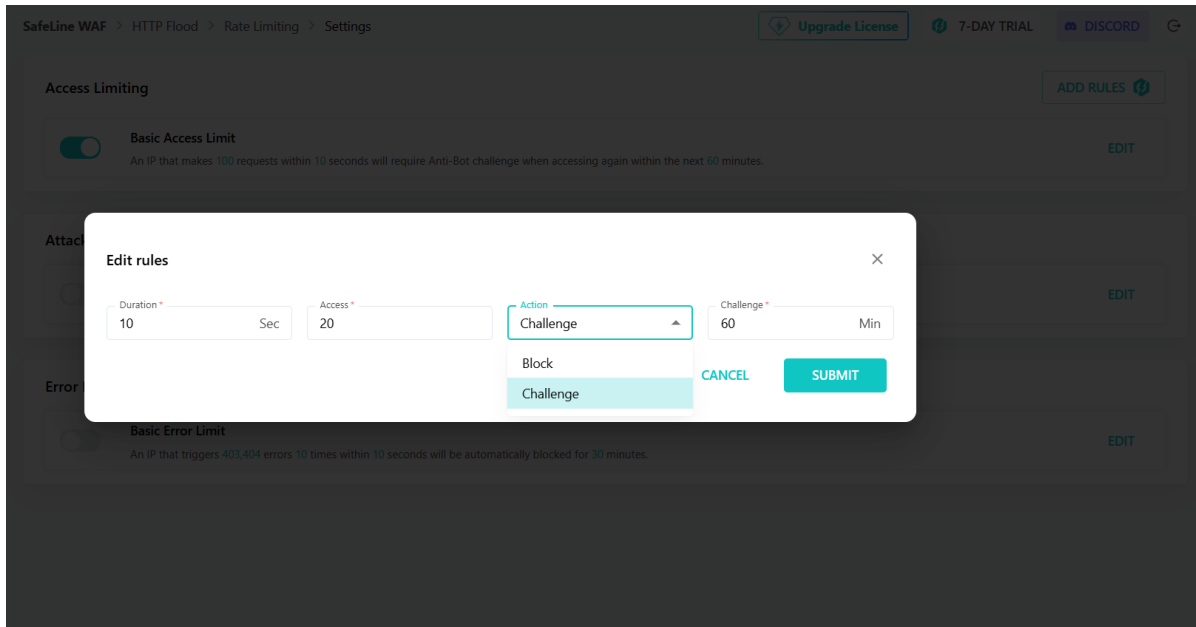


Figure 7. Rate-limit edit dialog during configuration. The challenge duration was shortened for the controlled lab test before execution.

7.1 Controlled traffic generation

```
ab -n 50 -c 5 https://dvwa.local:8443/DVWA/login.php
```

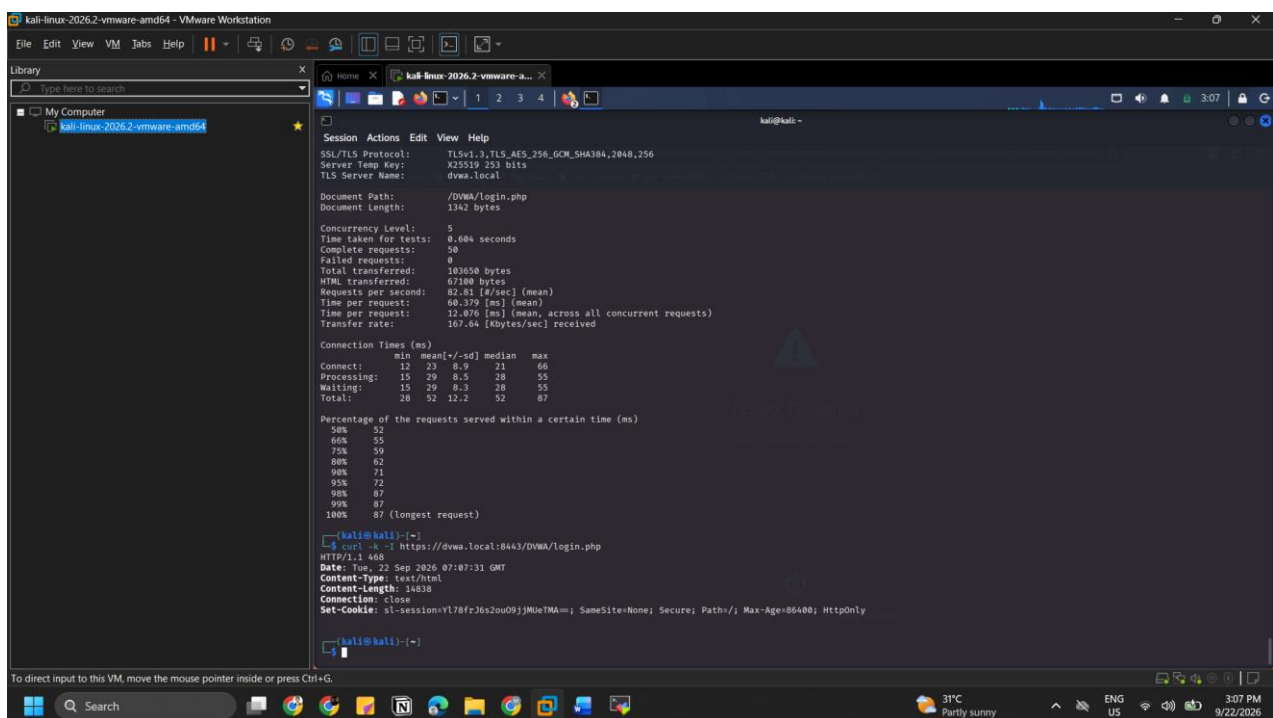


Figure 8. ApacheBench completed 50 requests at approximately 82.81 requests/second; a subsequent request received SafeLine HTTP 468 challenge behavior.

The ApacheBench run itself reported zero failed requests because the configured policy challenged subsequent access after the threshold was crossed. The next request received a SafeLine-specific 468 response and SafeLine session cookie. HTTP 468 is not a standard Internet status code; it is treated here only as observed SafeLine behavior in this lab.

7.2 Recovery validation

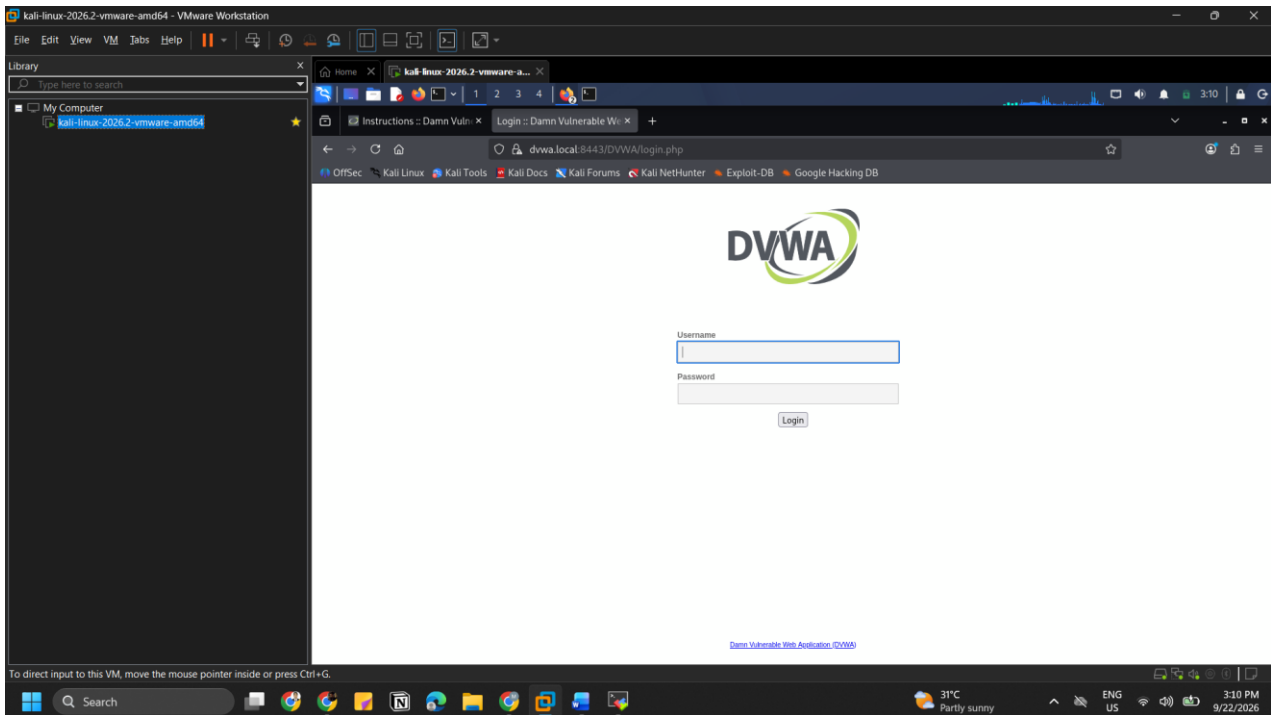


Figure 9. Normal DVWA login access restored after the configured SafeLine challenge window expired.

Result

PASS - SafeLine rate limiting detected a controlled request burst, enforced an Anti-Bot challenge, and automatically restored normal access after the configured period.

8. Security Validation: SafeLine Authentication

A SafeLine local user named labuser was created to demonstrate an authentication gateway in front of DVWA. This created a separate security layer before the DVWA application login.

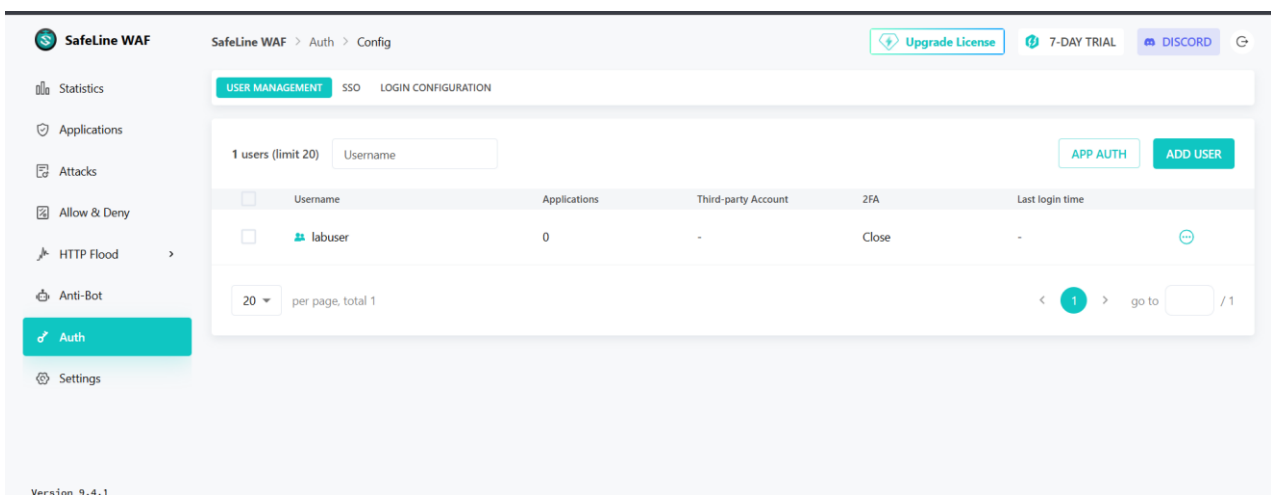


Figure 10. SafeLine Auth user management with the labuser account.

8.1 Sign-in gateway

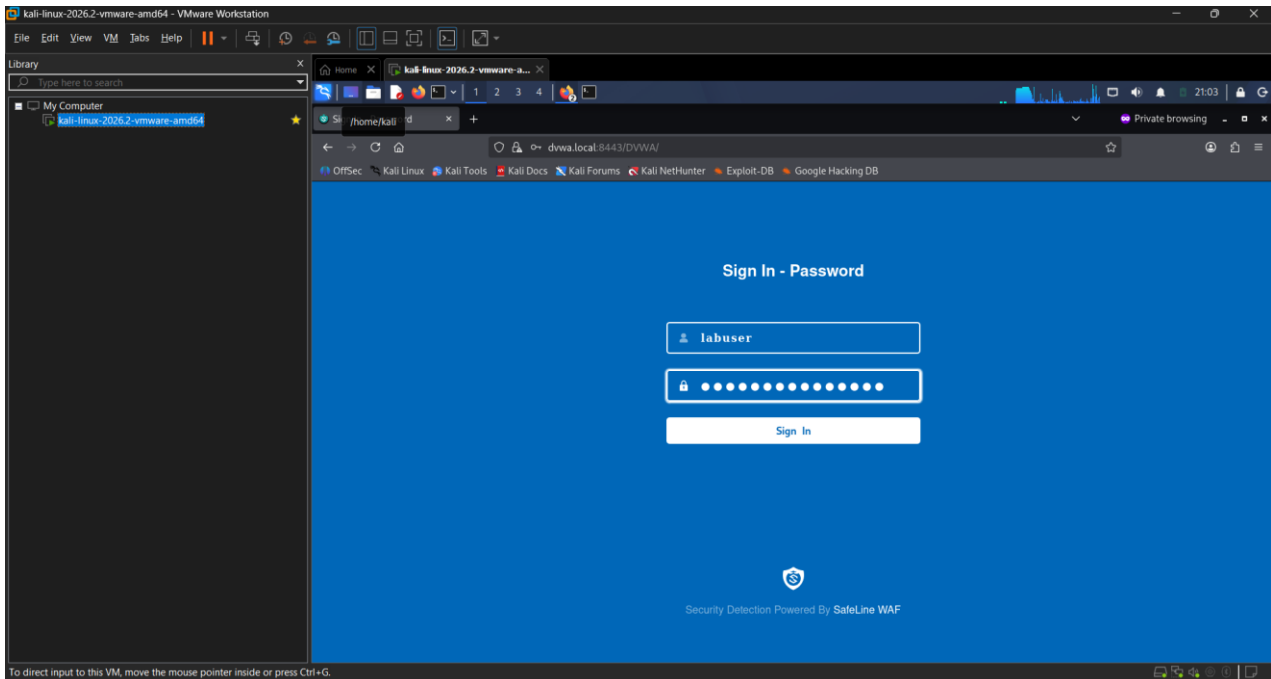


Figure 11. SafeLine password sign-in presented before the user can reach DVWA. Password characters are masked in the screenshot.

8.2 Authentication versus authorization

During testing, valid credentials initially produced an "Account not authorized" message. This was useful evidence that authentication and application authorization are distinct controls: the identity had successfully signed in, but it had not yet been approved for the DVWA application. After the application authorization was accepted, the user could proceed through SafeLine to DVWA.

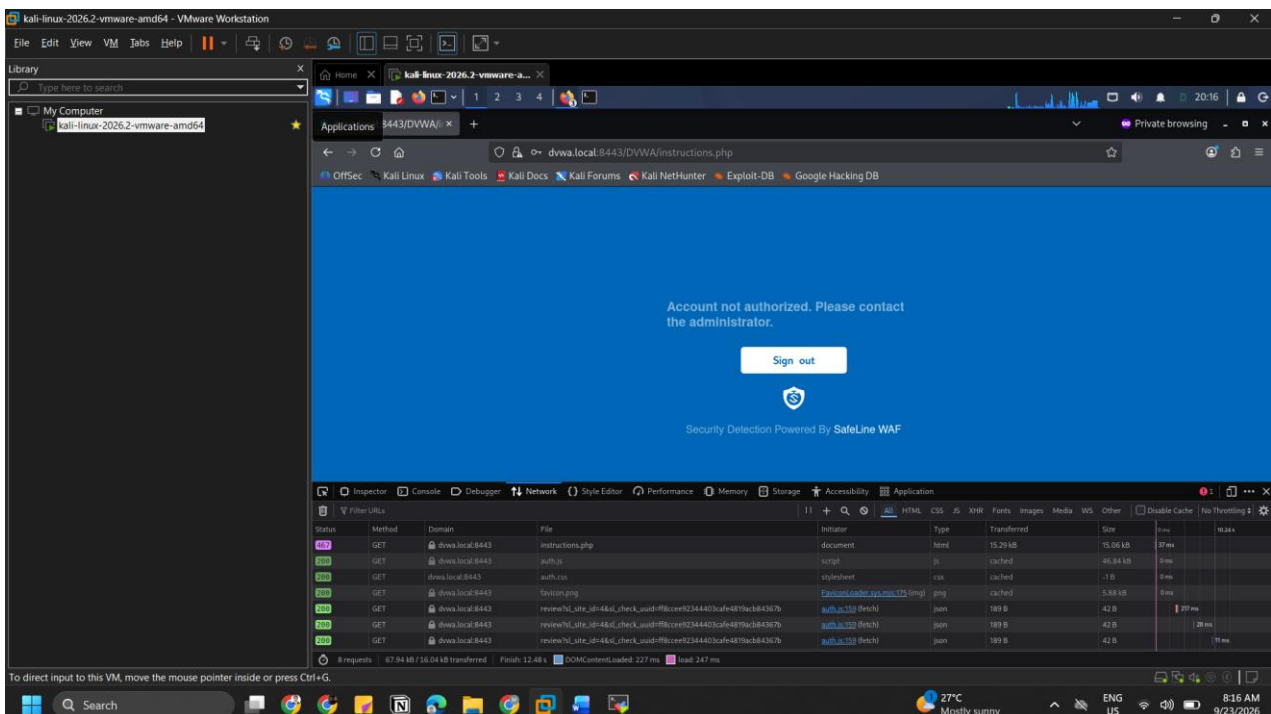


Figure 12. SafeLine authenticated the account but denied application access until authorization was approved.

Result

PASS - SafeLine enforced authentication before DVWA and separately enforced application authorization for the authenticated user.

9. Security Validation: Source-IP Deny Rule

A custom deny rule was applied to the Kali client IP to verify explicit source-IP blocking. Repeated refreshes increased the event hit count; the rule was later disabled and enforcement cleared as expected.

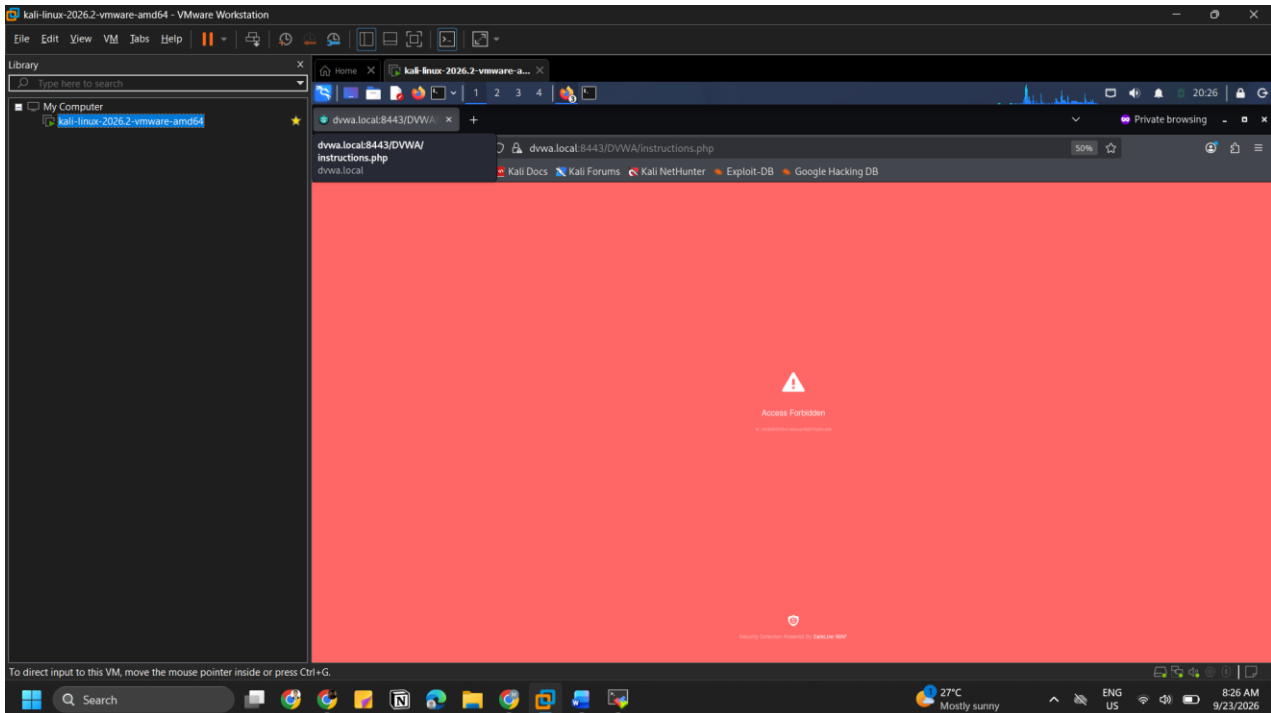


Figure 13. SafeLine Access Forbidden response while the Kali source-IP deny rule was active.

Hit Count	Duration
32	7 minutes Processing

Figure 14. Grouped deny event showing repeated hits while the rule was active.

Operational note

A grouped event marked Processing does not necessarily mean the client remains blocked indefinitely. The latest request and active control state should be checked. During testing, repeated browser refreshes increased the hit count and made the event appear active longer.

Result

PASS - SafeLine explicitly denied the Kali source IP and returned a WAF block page. Access returned after the rule/enforcement condition was cleared.

10. Backend Hardening: Closing the Direct :8080 Bypass

After the WAF controls were validated, the architecture still had a security gap: DVWA could be reached directly on 192.168.1.17:8080, allowing a LAN client to bypass SafeLine completely. This was treated as a hardening finding rather than left as a lab shortcut.

10.1 Network discovery

Docker inspection showed that safeline-tengine uses host networking. Apache was the actual host listener on TCP/8080. This made a loopback-only design practical because SafeLine can reach 127.0.0.1 in the host network namespace.

```
safeline-tengine    NETWORKS: host
```

Apache listener before hardening: *:8080

10.2 Remediation

1. SafeLine upstream was changed from the LAN address on port 8080 to http://127.0.0.1:8080/.
2. Apache ports.conf was changed from Listen 8080 to Listen 127.0.0.1:8080.
3. Apache configuration was validated with apache2ctl configtest before restart.
4. Listener state was rechecked with ss to confirm only 127.0.0.1:8080 was bound.

```

juanito@ubuntu:~$ sudo nano /etc/apache2/ports.conf
juanito@ubuntu:~$ sudo apache2ctl configtest
[Wed Sep 23 08:58:30.686689 2026] [core:crit] [pid 28457] (EAI 2)Name or service not known: AH00077: alloc_listener: failed to set up sockaddr for 127.0.0.1
AH00526: Syntax error on line 5 of /etc/apache2/ports.conf:
Listen setup failed
juanito@ubuntu:~$ sudo nano /etc/apache2/ports.conf
juanito@ubuntu:~$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK
juanito@ubuntu:~$ sudo systemctl restart apache2
juanito@ubuntu:~$ sudo ss -ltnp | grep ':8080'
LISTEN 0      511          127.0.0.1:8080      0.0.0.0:*        users:((("apache2",pid=28598,fd=3),("apache2",pid=28597,fd=3),("apache2",pid=28596,fd=3),("apache2",pid=28595,fd=3),("apache2",pid=28594,fd=3),("apache2",pid=28592,fd=3)))
juanito@ubuntu:~$ curl -I http://127.0.0.1:8080/DVWA/login.php
HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 00:59:36 GMT
Server: Apache/2.4.58 (Ubuntu)
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=0mo4ctioio0g3be6ks9khvejf; expires=Thu, 24 Sep 2026 00:59:36 GMT; Max-Age=86400; path=/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=5hk9509llvn454ta8trja1uqi1; expires=Thu, 24 Sep 2026 00:59:36 GMT; Max-Age=86400; path=/; domain=127.0.0.1; HttpOnly; SameSite=Strict
Content-Type: text/html; charset=utf-8

```

Figure 15. Ubuntu validation after hardening: Apache listens on 127.0.0.1:8080 and serves DVWA locally with HTTP 200.

10.3 Bypass closure validation

```

kali@kali:~$ curl -I http://127.0.0.1:8080/DVWA/login.php
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Could not connect to server

kali@kali:~$ curl -k -I https://dvwa.local:8443/DVWA/login.php
HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 00:59:50 GMT
Content-Type: text/html
Content-Length: 15056
Connection: close
Set-Cookie: sl_auth_info=fd94e67c50454aada66f64110b381607_4_5_1; SameSite=None; Secure; Path=/; Max-Age=3600
Set-Cookie: sl-session=1JiQXwZ2tGokInUCu9w02g==; SameSite=None; Secure; Path=/; Max-Age=86400; HttpOnly

kali@kali:~$ curl -I --connect-timeout 5 \
http://192.168.1.17:8080/DVWA/login.php
curl: (7) Failed to connect to 192.168.1.17 port 8080 after 26 ms: Could not connect to server

```

Figure 16. Kali can no longer connect directly to 192.168.1.17:8080, while the SafeLine-protected endpoint continues to respond through the authentication layer.

Security improvement

Before: any LAN client could connect directly to DVWA on 192.168.1.17:8080 and bypass the WAF. After: Apache accepts only loopback traffic on 127.0.0.1:8080, so clients must use the SafeLine-protected path.

11. Final Validation Matrix

Control / Test	Test Path	Observed Result	Status
DVWA backend health	Ubuntu -> 127.0.0.1:8080	HTTP 200	PASS
Kali hostname resolution	dvwa.local -> 192.168.1.17	Resolves correctly	PASS
Protected HTTPS endpoint	Kali -> dvwa.local:8443	SafeLine responds	PASS
SQL injection prevention	DVWA SQLi module through SafeLine	Access Forbidden + SQL Inj logs	PASS
Attack logging	SafeLine Attacks -> Logs	Kali IP and SQL Inj events visible	PASS
HTTP flood / rate limit	50 requests, concurrency 5	SafeLine challenge enforced	PASS
Rate-limit recovery	After challenge window	Normal DVWA access restored	PASS
Authentication gateway	Unauthenticated protected request	SafeLine sign-in shown	PASS
Application authorization	Authenticated but unapproved user	Authorization denied until approved	PASS
Source-IP deny	Kali IP denied	Access Forbidden	PASS

Control / Test	Test Path	Observed Result	Status
Direct backend bypass	Kali -> 192.168.1.17:8080	Connection fails	PASS
Loopback backend	Ubuntu -> 127.0.0.1:8080	HTTP 200	PASS
Standard 443 URL	https://dvwa.local	Still owned by Traefik	PLANNED

12. Security Findings and Lessons Learned

Finding 1 - Port ownership matters

The 404 was generated by Traefik, not DVWA. TLS certificate identity was more useful than the HTTP status alone for identifying the component handling the request.

Finding 2 - DNS correctness does not prove routing correctness

Adding dvwa.local to Kali /etc/hosts fixed name resolution, but requests still landed on the wrong HTTPS listener until the port conflict was isolated.

Finding 3 - A WAF is ineffective if the origin is directly reachable

The original :8080 backend exposure allowed a straightforward SafeLine bypass. Loopback-only binding materially improved the design.

Finding 4 - Authentication and authorization are separate

Valid SafeLine credentials did not automatically authorize the user to access DVWA. The lab demonstrated both identity verification and per-application authorization.

Finding 5 - Rate limiting needs controlled thresholds

Short challenge durations and modest request volumes are appropriate for homelab validation. Aggressive thresholds can lock out the testing client and create noisy logs.

Finding 6 - Non-standard HTTPS ports are not inherently less secure

8443 can carry the same TLS protections as 443. The difference is convention, compatibility, and URL cleanliness, not inherent cryptographic strength.

13. Recommendations and Future Enhancements

13.1 Near-term improvement: clean standard HTTPS URL

Keep Traefik on TCP/443 and route only dvwa.local to SafeLine using Traefik TLS passthrough. Traefik would use SNI only to select the backend while SafeLine remains the component that terminates TLS and performs WAF inspection.

```

Kali -> https://dvwa.local:443
      |
      v
Traefik :443 (SNI routing / TLS passthrough)
      |
      v
SafeLine :8443 (TLS termination + WAF)
      |
      v
127.0.0.1:8080 Apache / DVWA

```

13.2 Additional hardening

- Restrict SafeLine management port 9443 to trusted admin hosts or an admin network where practical.
- Use a small internal CA or trusted local certificate workflow so lab browsers can validate dvwa.local without bypassing certificate checks.

- Back up SafeLine, Apache, and Traefik configuration before major networking changes.
- Keep direct backend services bound to loopback or private container networks whenever a reverse proxy/WAF is meant to be the only ingress path.
- Keep Kali, Ubuntu, DVWA, SafeLine, Docker, and the homelab stack patched while preserving intentionally vulnerable application behavior only inside the isolated lab.

13.3 Detection and portfolio extensions

- Forward SafeLine, Apache, and system logs into a SIEM or centralized logging stack for correlation.
- Integrate WAF metrics and security events into the existing Prometheus/Grafana monitoring environment where appropriate.
- Repeat the attack -> detect -> block -> log -> recover method for XSS, command injection, file inclusion, and other DVWA modules.
- Add OWASP Juice Shop as a second intentionally vulnerable application to broaden web-security testing.
- Document configuration as code where possible and add architecture/evidence diagrams to the GitHub repository.
- Consider a future dedicated SafeLine VM/IP if additional hardware or compute becomes available; this would let SafeLine own TCP/443 independently of Traefik.

14. Project Closure

The DVWA + SafeLine WAF homelab is complete for the defined current scope. The lab successfully demonstrates application deployment, reverse-proxy/WAF integration, root-cause troubleshooting, SQL injection prevention, rate limiting, Anti-Bot behavior, authentication, authorization, source-IP blocking, and origin hardening.

The most important final architectural improvement was removing direct LAN exposure of the DVWA backend. SafeLine is no longer just an optional protected route; the backend is loopback-only, forcing normal client access through the security layer.

Final status: COMPLETE

The environment is suitable for portfolio documentation and continued controlled experimentation. The Traefik TLS-passthrough clean-URL design and broader SIEM/attack coverage are recommended future iterations, not unresolved failures.

Appendix A - Key Commands and Evidence

A.1 DNS / resolution

```
# Kali
sudo nano /etc/hosts
# 192.168.1.17    dvwa.local

getent hosts dvwa.local
ping -c 2 dvwa.local
```

A.2 Direct and protected endpoint checks

```
# Historical direct backend test (before hardening)
curl -I http://192.168.1.17:8080/DVWA/login.php

# Protected endpoint
curl -k -I https://dvwa.local:8443/DVWA/login.php
```

A.3 Traefik root-cause evidence

```
curl -k -v https://dvwa.local/DVWA/login.php
# Key evidence: subject: CN=TRAEFIK DEFAULT CERT
# Result: HTTP/2 404
```

A.4 Working SafeLine listener evidence

```
curl -k -v https://dvwa.local:8443/DVWA/login.php
# Key evidence: CN=dvwa.local
# Result: HTTP/1.1 200 OK
# SafeLine cookie: sl-session=...
```

A.5 HTTP flood test

```
ab -n 50 -c 5 https://dvwa.local:8443/DVWA/login.php

# Follow-up enforcement check
curl -k -I https://dvwa.local:8443/DVWA/login.php
```

A.6 Apache origin hardening

```
sudo grep -RniE 'Listen.*8080|VirtualHost.*8080' \
/etc/apache2/ports.conf \
/etc/apache2/sites-enabled \
/etc/apache2/sites-available

# Final listener in /etc/apache2/ports.conf
Listen 127.0.0.1:8080

sudo apache2ctl configtest
sudo systemctl restart apache2
sudo ss -lntp | grep ':8080'
```

A.7 Final origin validation

```
# Ubuntu - must work
curl -I http://127.0.0.1:8080/DVWA/login.php

# Kali - must fail after hardening
curl -I --connect-timeout 5 http://192.168.1.17:8080/DVWA/login.php
```

```
# Kali - protected path remains available through SafeLine  
curl -k -I https://dvwa.local:8443/DVWA/login.php
```

Appendix B - Final Protected Access State

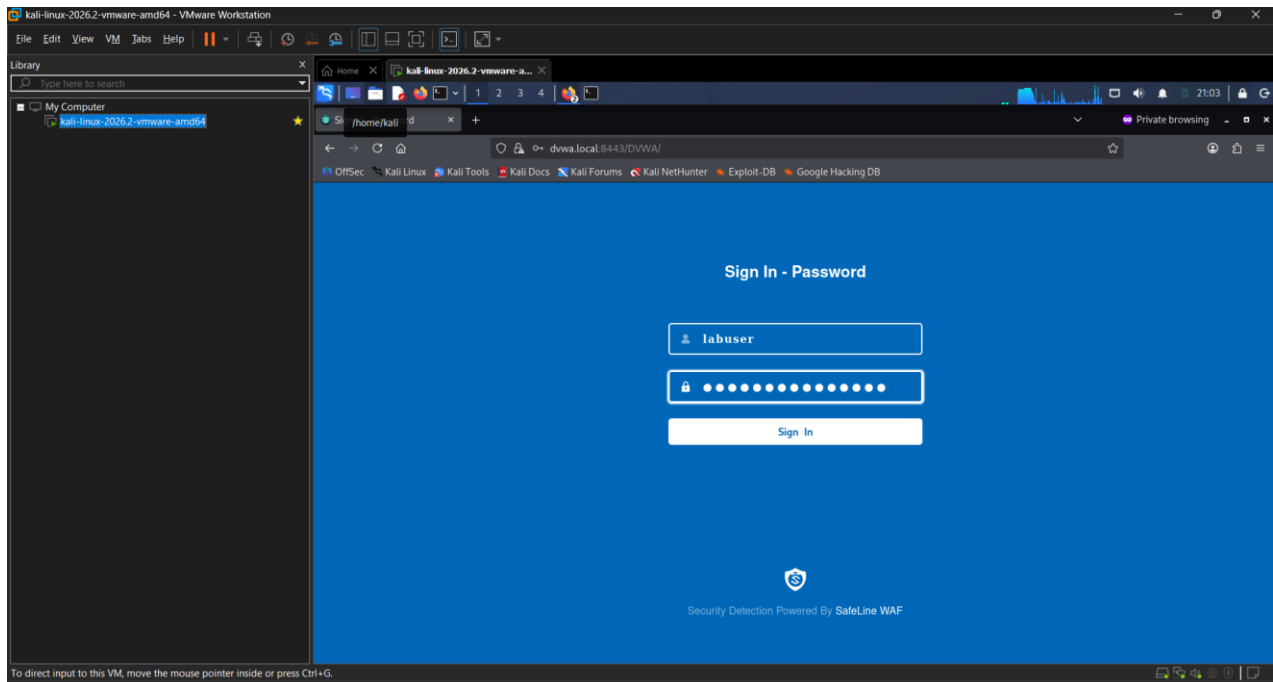


Figure 17. Final protected access state: SafeLine authentication is presented on dvwa.local:8443 before DVWA.